

ML Specification Report — playground-series-s3e11

Media Campaign Cost Prediction • AIDE agent (20-step tree search, Claude Sonnet via shim)

Figures grounded in the run's `facts_aide.json`, the agent's own `report.md` and `best_solution.py` (run `2-complex-thundering-silkworm`), its `journal.json` tree, and `config.yaml` for task metadata. Dataset shapes were measured directly from the run's own `input/` copy of the competition files.

Contents	
Media Campaign Cost Prediction • AIDE agent (20-step tree search, Claude Sonnet via shim)	1
Overview	1
Data	2
Models & Architecture	3
Training procedures	4
The search trajectory	4
Specification fields	5
Inference procedures	6
Evaluation & Benchmarking	6

Overview

The task is to predict the cost of a store media campaign for 240,224 test rows (metric: RMSLE, minimized). AIDE solves it not by executing a designed pipeline but by **searching** for one: it drafts a candidate solution, runs it, reads the traceback or the printed CV score, and mutates the best node it has seen. Over **20 steps** it produced **13 scored solutions and 7 buggy ones**, moving from a first LightGBM draft at **CV RMSLE 0.29666** to a champion at **CV RMSLE 0.2941** — a seed-bagged CatBoost with native categorical handling, found at **step 14**.

$$0.29666 - 0.29410 = 0.00256$$

That champion is the strongest of the three agents on this competition. Its **private LB score is 0.29445** (73.3rd percentile, rank 256/954), ahead of both our own from-scratch agent (0.29597) and the NVIDIA reproduce-agent (0.29624); `facts_aide.json` records `local_winner = aide` and `lb_winner = aide`. The win came from a single architectural discovery — CatBoost's ordered target statistics on the low-cardinality columns — that AIDE reached only after a crash and two failed detours, and it was bought at

a steep price in compute: four of the seven failures were 30-minute timeouts that consumed roughly three quarters of the run's wall clock.

Why it matters. Forecasting campaign cost helps retailers budget promotions and allocate marketing spend efficiently across stores and periods. Methodologically, this episode is a clean test of whether an unguided search agent can find a model-family switch that a hand-built pipeline might fix in advance.

Data

Purpose of Data. Predict the media-campaign `cost` for each store record — a continuous-target **regression** Playground Series episode scored by RMSLE, minimized (`config.yaml`: `evaluation_metric: rmsle`, `optimization_direction: minimize`, `target_column: cost`). **Data Format** is clean **tabular CSV**, entirely numeric. **Data Volume**, measured from the run's `input/` directory, is **360,336 training rows × 17 columns / 240,224 test rows × 16 columns**, leaving **15 raw predictive features** once `id` and `cost` are removed.

Data Quality is high and required no repair: **zero missing values** in either split and **zero duplicate rows** once `id` is excluded. AIDE never ran a dedicated EDA step — the AIDE loop has no separate exploration phase, so every property of the data it exploited had to be inferred from the column names and from what the executed code printed back. This produced one clear factual error worth surfacing: **the agent's own `report.md` opens by asserting that "the target exhibits a right-skewed, discrete-like distribution typical of cost/spend data", but the measured target is near-symmetric** — `cost` $\in$ [50.79, 149.75], mean 99.6147, skew 0.0191, with 328 distinct values. The "discrete-like" half of the claim is defensible (328 levels over 360k rows); the "right-skewed" half is not. The `log1p` transform AIDE applied is nevertheless correct, but for a different reason than the one it gave: RMSLE is RMSE on `log1p`, so modelling in log space aligns the training objective with the metric regardless of skew.

The property AIDE did find — empirically, through the search rather than through analysis — is that most of this dataset's columns behave like **categories rather than magnitudes**. Applying the champion's own selection rule (combined train+test cardinality ≤ 25) to the 20-column feature matrix marks **14 of 20 features as categorical**, including `store_sqft` (20 distinct values), the five amenity flags, and the household-count columns; only six columns — `store_sales`, `gross_weight`, `units_per_case` and three engineered ratios — are left as true continuous numerics. Treating those 14 as categoricals rather than as ordered numbers is the whole of AIDE's advantage on this problem.

Annotation Guidelines. The label is `cost`, a continuous float. Submissions are predicted campaign costs scored by RMSLE — no thresholding or class decision is involved. AIDE did test **snapping predictions to the nearest observed training value** (step 3), exploiting the discreteness; it produced no change (0.29658 before and after) and was dropped.

Feature Set. From the 15 raw columns the champion builds **20 features**:

Group	Features
Raw	the 15 original numeric columns
Counts / differences	amenities_count (sum of 7 binary flags), children_diff (total_children - num_children_at_home)
Ratios	sales_per_unit , weight_per_case , sqft_per_car

Of these 20, **14 are cast to strings and handed to CatBoost as native categoricals**; the remaining 6 stay `float64`. Notably, AIDE also tried the alternative encoding route — **K-fold target encoding**, both per-column (step 4, 0.29663) and as a composite concatenated group key over all near-categorical columns (step 5, 0.31388) — and both underperformed. The composite key was the single worst scored node of the run.

Splitting strategy. A single **5-Fold KFold (shuffle, random_state=42)**, held fixed across every node of the search, with OOF RMSLE computed on the original scale. This constancy is what makes the 13 scored nodes comparable to one another. The run was **submitted to Kaggle**, so unlike a CV-only run both public and private leaderboard scores exist: **public 0.29389, private 0.29445**.

Models & Architecture

Purpose of Architecture. A regressor that minimizes RMSLE on media-campaign cost. **Architecture Type** is a **single gradient-boosted decision-tree model, seed-bagged** — specifically CatBoost with ordered target statistics over 14 categorical features, fitted twice per fold under different random seeds and averaged. It is deliberately not an ensemble across model families: AIDE arrived at the champion by **deleting** the LightGBM and XGBoost members from a 3-way blend once it saw CatBoost alone was stronger (step 10, plan: "the atomic improvement is to drop the LGB/XGB models").

Input Format is a 20-column mixed matrix — 6 float columns plus 14 string-cast categorical columns — with the target modelled as `log1p(cost)`. **Input Dimension** is 20 features per row, a flat vector with no spatial or sequence structure.

Architecture Description. Each member is a symmetric (oblivious) boosted tree ensemble grown by CatBoost with `depth = 10`, `learning_rate = 0.03`, `l2_leaf_reg = 6.0`, `bagging_temperature = 0.5`, `loss_function = RMSE` on the log target, up to `iterations = 4000` with `early_stopping_rounds = 100` against the fold's own validation split and `use_best_model=True`. The categorical columns are handled by CatBoost's internal ordered target-statistic (CTR) machinery at default `max_ctr_complexity`, which builds feature combinations automatically — this is the mechanism that AIDE's manual pairwise-interaction attempt (step 11) tried and failed to beat by hand.

Model Complexity is best expressed as members $\times$ trees $\times$ depth rather than a dense parameter count: **2 seeds $\times$ 5 folds = 10 fitted CatBoost models**, each up to 4,000 depth-10 oblivious trees before early stopping, with inference averaging the 2 seeds within a fold and then the 5 folds. There is

no blend layer, no stacker, and no learned combination weights — the only aggregation is an unweighted mean.

Training procedures

The search trajectory

AIDE's `journal.json` records a genuine tree, not a chain: **5 independent root drafts (steps 0, 1, 5, 6, 7)** and 15 nodes that improve or debug a named parent. The champion sits at depth 5 along the path **0 → 2 → 8 → 9 → 10 → 14**. Thirteen nodes returned a score; seven crashed.

Step	Parent	What AIDE tried	CV RMSLE
0	—	LightGBM on <code>log1p(cost)</code> , engineered features (first draft)	0.29666
1	—	XGBoost <code>hist</code> , same features (second draft)	0.29676
2	0	weighted LGB+XGB blend, grid-searched weight	0.29658
3	2	+ snap predictions to nearest observed <code>cost</code>	0.29658
4	2	+ per-column K-fold target encoding	0.29663
5	—	composite group-key target encoding (draft)	0.31388
6	—	PyTorch model (draft)	<code>ModuleNotFoundError</code>
7	—	PyTorch model, second attempt (draft)	<code>ModuleNotFoundError</code>
8	2	add CatBoost as a third blend member	<code>CatBoostError</code>
9	8	fix: pass a <code>DataFrame</code> , not a <code>float64</code> array, with <code>cat_features</code>	0.29437
10	9	drop LGB/XGB, CatBoost only, expanded categorical set	0.29414
11	10	+ explicit pairwise interaction columns, <code>max_ctr_complexity=3</code>	0.29445
12	7	<code>HistGradientBoostingRegressor</code> fallback for the missing torch	0.29722
13	6	<code>HistGradientBoostingRegressor v2</code> , label-encoded	0.29743
14	10	seed-bagging: 2 seeds × 5 folds, averaged (champion)	0.2941
15	14	richer categorical combinations	<code>TimeoutError</code>
16	14	Tweedie loss	<code>TimeoutError</code>
17	16	fix the CTR-combination hang	<code>TimeoutError</code>
18	17	Tweedie with <code>max_ctr_complexity=1</code> , depth 6	0.30018
19	15	reduce the fold × seed × iteration budget	<code>TimeoutError</code>

Read as a search, the run has three distinct phases. **Steps 0–5 are a plateau**: five different framings of the same idea — boosted trees on ordinally-encoded features — all land within 0.0002 of each other (0.29658–0.29676), except the composite target-encoding draft, which is much worse. AIDE's own plan

text registers this explicitly from step 2 onward ("both LightGBM and XGBoost single-model approaches plateaued around 0.2966–0.2968"), and it correctly concluded that the plateau was structural rather than a tuning problem.

Steps 6–9 are the escape, and it happens through a crash. Two consecutive drafts reach for PyTorch, which is not installed, and die instantly at import (`exec_time` 0.0018 s and 0.0009 s — the two cheapest failures of the run). Step 8 then adds CatBoost to the blend and crashes with a `CatBoostError` because a homogeneous `float64` array was passed alongside `cat_features`. The step-9 repair — pass a DataFrame with the categorical columns string-cast — is the moment the run turns:

```
0.29658 - 0.29437 = 0.00221
```

A single bug-fix node delivered 86% of the run's total improvement:

```
0.00221 / 0.00256 = 0.863
```

Steps 10–19 are refinement and then attrition. Dropping the weaker members (0.29414) and seed-bagging (0.2941) extract the last of the gain, and every subsequent step fails to beat step 14. Four of the last five steps time out, and the one that completes — Tweedie loss under a constrained `max_ctr_complexity` — scores 0.30018, the second-worst result of the run. The final six steps of the budget produced no improvement at all.

The cost of this is the run's most uncomfortable number. Total execution time was **9,362.3 s (2.60 h)** across 20 steps, mean 468.1 s, max 1,800.4 s:

```
9362.3 / 3600 = 2.60
```

The four `TimeoutError` steps each ran to the harness ceiling visible in `exec_time_max_s = 1800.4`:

```
4 × 1800.4 = 7201.6  
7201.6 / 9362.3 = 0.769
```

About 77% of the run's wall clock was spent on four steps that returned nothing, and the overall failure rate was:

```
7 / 20 = 0.35
```

AIDE diagnosed the timeout cause correctly (its step-17 and step-19 plans identify "depth=10 CatBoost models with 14 categorical features ... combined with 5 folds × 2 seeds × 4000 iterations" as the bottleneck) but never recovered a scoring node from it — it kept proposing more expensive variants of an already budget-saturated configuration rather than trimming first and enriching second.

Specification fields

The **Loss Function** is RMSE on the `log1p(cost)` target (CatBoost `loss_function="RMSE"`, `eval_metric="RMSE"`) — directly equal to the competition RMSLE — with predictions passed through

`expm1` and clipped at ≥ 0 . The **Optimization Algorithm** is gradient boosting over symmetric trees with ordered target statistics — **not** SGD/ADAM. The **Learning Rate** is the boosting shrinkage, `0.03`. There is **no Learning Rate Scheduler** (N/A — GBDT convergence is governed by per-fold early stopping at 100 rounds against the validation split, not a schedule) and **no Batch Size** (N/A — full-dataset histogram boosting, not mini-batched).

Training Duration is 9,362.3 s of execution across the whole search; the champion node itself took **786.5 s** (from `journal.json`), the third most expensive scoring step of the run. **Training Memory** was **not recorded** — AIDE does not instrument peak RSS, and no memory cap was configured. **Transfer Learning** is N/A (no pretrained weights); AIDE's analogue is its **journal-conditioned prompt** — each new node is drafted with the full results history of prior nodes in context, which is how "results history" reasoning appears verbatim in the later plans, and it is the only mechanism by which experience propagates. **Data Augmentation** is N/A (tabular); the analogues here are the five engineered features and the two-seed bagging.

Reproducibility Standards are partial. Fold seed is fixed (`KFold(shuffle=True, random_state=42)`) and the bagging seeds are explicit (`[42, 2023]`), so the champion script is re-runnable as written. What is not reproducible is the search itself: node selection depends on LLM sampling, so a re-run of the 20 steps would produce a different tree. There is no replay gate and no digit-for-digit verification of cached nodes — the only artifact carried forward is `best_solution.py`, and `facts_aide.json` records that the submitted file came from a **direct rerun of the best node** rather than from a cached prediction array.

Inference procedures

Decision Threshold is **N/A** — RMSLE is a regression metric, so the raw predicted cost is submitted.

Post-processing is minimal by the end: `expm1` back-transform, `np.clip(..., a_min=0)`, then a reindex onto `sample_submission.csv`'s `id` order. The one non-trivial post-processing idea AIDE tested — snapping to the nearest observed training `cost` — was measured (step 3) and discarded for zero gain. Prediction averaging happens in log space across the 2 seeds and then in original space across the 5 folds. **Inference Duration** was **not recorded** separately; AIDE times whole scripts, not their scoring phase. **Inference Memory** is likewise **not recorded**.

Evaluation & Benchmarking

Task for Performance Evaluation. Predict media-campaign `cost` for every test row and minimize RMSLE.

Performance Metrics. The champion reaches **CV RMSLE 0.2941** locally and **0.29445 on the private leaderboard**, with **public 0.29389**. The local estimate proved close to honest — the CV-to-private drift is small and in the pessimistic-CV direction:

$$0.29445 - 0.29410 = 0.00035$$

and the public/private spread is likewise tight:

$$0.29445 - 0.29389 = 0.00056$$

Placement is **rank 256/954, 73.3rd percentile**. The scored trajectory across the run was $0.29666 \rightarrow 0.29658 \rightarrow 0.29437 \rightarrow 0.29414 \rightarrow \mathbf{0.2941}$, with the decisive move being a model-family switch rather than tuning or ensembling.

Performance Benchmarking. All three agents attacked the same competition independently; the private-LB figures below are the only cross-lane numbers used in this report.

Agent	Approach	Private LB RMSLE	Note
AIDE	20-step tree search $\rightarrow$ seed-bagged categorical-native CatBoost	0.29445	winner
Our agent	from-scratch GBDT pool + tree-search blend	0.29597	behind
NVIDIA	reproduce-agent distilling a public ensemble kernel	0.29624	behind

AIDE wins this competition on both the local metric and the leaderboard:

$$\begin{aligned} 0.29597 - 0.29445 &= 0.00152 \\ 0.29624 - 0.29445 &= 0.00179 \end{aligned}$$

The margins are small in absolute terms, but they are consistent in direction and they trace to one identifiable decision. Both comparator lanes treat this dataset's low-cardinality columns as ordered numerics and then spend their effort on ensembling and hyperparameter search; AIDE, by accident of a crashed blend node, ended up handing them to CatBoost as categoricals and let its CTR machinery build the interactions. On a dataset whose signal lives almost entirely in store-profile combinations, that representation choice beat both a tuned pool and a reproduced public kernel.

Two caveats belong on the record. First, **AIDE's efficiency here was poor even though its accuracy was best** — 2.60 hours of compute, 35% of steps failing, 77% of wall clock burned on timeouts, and the last six steps producing nothing. Second, **the discovery was not planned**: reading the journal, CatBoost entered the search as a third blend member, crashed on a dtype mismatch, and only became the champion because the repair incidentally introduced native categorical handling. A search agent that had not crashed at step 8 might never have found it.